

MVP - Data Engineering

Pokémon Challenge - Weedle's Cave:

Um estudo de manipulação de dados de batalhas pokémon

Aluno: Guilherme Farias Alves

Email: guilhermefarias@gmail.com

GitHub: <https://github.com/Fariasgui>

Sprint: Engenharia de Dados (40530010057_20260_01)

MVP - Data Engineering	0
1. Contexto de Negócio e Perguntas	2
1.1. Contextualização do Projeto	2
1.2. Objetivo do MVP	2
1.3. Perguntas de Negócio	3
1.4. Fonte dos Dados	3
2. Carga dos Dados	4
2.1. Ambiente de Desenvolvimento	4
2.2. Ingestão dos Arquivos	5
2.3. Construção da Camada Bronze	6
2.5. Evidências da Carga	7
3. Modelagem e Catálogo de Dados	9
3.1. Arquitetura Medallion	9
3.2. Modelo de Dados da Camada Gold	10
3.3. Relacionamentos e Granularidade	11
3.4. Catálogo de Dados — dim_pokemon	12
3.5. Catálogo de Dados — fact_battles	13
3.6. Catálogo de Dados — agg_pokemon_performance	14
3.7. Evidências da Modelagem	14
4. Pipeline de Dados	16
4.1. Visão Geral do Pipeline	16
4.2. Pipeline Source → Bronze	17
4.3. Pipeline Bronze → Silver	17
4.4. Pipeline Silver → Gold	22
4.5. Organização dos Notebooks e Scripts	25
5. Qualidade dos Dados	26

5.1. Estratégia de Qualidade	26
5.2. Qualidade da Entidade Pokémon	27
5.3. Qualidade dos Combates	28
5.4. Integridade Referencial	29
5.6. Resultados das Validações	31
6. Análise dos Dados	32
6.1. Metodologia da Análise	32
6.2. Distribuição do Número de Batalhas	32
6.3. Pergunta 1 — Speed × Taxa de Vitória	33
6.4. Pergunta 2 — Atributos de Vencedores × Perdedores	34
6.5. Pergunta 3 — Legendary × Não Legendary	35
6.6. Pergunta 4 — Taxa de Vitória por Geração	36
6.7. Pergunta 5 — Pokémon com Maiores Taxas de Vitória	37
6.8. Discussão Geral dos Resultados	39
7. Autoavaliação	39
7.1. Resultados Alcançados	39
7.2. Dificuldades Encontradas	40
7.3. Limitações do Projeto	41
7.4. Trabalhos Futuros	41
7.5. Considerações Finais	42

1. Contexto de Negócio e Perguntas

1.1. Contextualização do Projeto

O universo Pokémon possui diferentes espécies de criaturas caracterizadas por atributos como pontos de vida (HP), ataque, defesa, ataque especial, defesa especial e velocidade, além de características categóricas como tipo, geração e classificação como Pokémon Legendary. Em batalhas, a combinação dessas características pode estar associada ao desempenho e ao resultado obtido por cada Pokémon.

Este projeto utiliza o conjunto de dados **Pokémon Challenge Weedle's Cave**, composto por informações cadastrais e atributos dos Pokémon, além de registros históricos de batalhas entre eles.

Apesar de ser um *dataset* voltado para **Ciência de Dados**, neste projeto, o foco deixa de estar na construção do modelo preditivo e passa para a **Engenharia de Dados**, buscando desenvolver uma infraestrutura capaz de receber os dados brutos, armazená-los, tratá-los, validá-los e disponibilizá-los de maneira estruturada para consumo analítico..

1.2. Objetivo do MVP

O objetivo deste MVP é desenvolver um **pipeline de dados ponta a ponta em ambiente cloud**, utilizando o Databricks como plataforma de processamento e armazenamento.

A solução foi estruturada de acordo com os princípios da **arquitetura Medalhão (Medallion Architecture)**, dividindo o fluxo de dados em três camadas principais:

- **Bronze:** responsável pela ingestão e preservação dos dados provenientes dos arquivos de origem;
- **Silver:** responsável pela padronização, tipagem e validação da qualidade dos dados;
- **Gold:** responsável pela disponibilização dos dados modelados e preparados para consumo analítico.

O fluxo desenvolvido pode ser resumido da seguinte forma:

Arquivos CSV → Bronze → Silver → Gold → Análise

Além da construção das diferentes camadas, o projeto contempla verificações de qualidade, integridade referencial, modelagem dos dados e desenvolvimento de consultas analíticas em SQL.

Ao final do pipeline, espera-se disponibilizar uma camada Gold que permita responder às perguntas de negócio estabelecidas sem a necessidade de acessar ou manipular diretamente os arquivos de origem ou as tabelas intermediárias da camada Silver.

1.3. Perguntas de Negócio

Para orientar a construção do pipeline e da camada analítica, foram definidas cinco perguntas a serem respondidas a partir dos dados históricos de batalhas:

1. **Pokémon com maior atributo de velocidade (Speed) apresentam maior taxa de vitória?**
2. **Quais atributos apresentam as maiores diferenças entre Pokémon vencedores e perdedores?**
3. **Pokémon classificados como Legendary apresentam taxa de vitória diferente dos Pokémon não Legendary?**
4. **Como a taxa de vitória varia entre as diferentes gerações de Pokémon?**
5. **Quais Pokémon apresentam as maiores taxas de vitória considerando um número mínimo relevante de batalhas?**

Essas perguntas foram utilizadas como direcionadores para as decisões de modelagem da camada Gold e para a definição das métricas disponibilizadas para análise.

1.4. Fonte dos Dados

O projeto utiliza o conjunto de dados **Pokémon Challenge Weedle's Cave**, formado por três arquivos no formato CSV: `pokemon.csv`, `combats.csv` e `tests.csv`.

O arquivo `pokemon.csv` contém os dados cadastrais e os principais atributos de cada Pokémon. Entre as informações disponíveis estão seu identificador, nome, tipos, atributos de batalha, geração e indicação de pertencimento ao grupo Legendary.

Sua estrutura original contém os seguintes campos:

`#`, `Name`, `Type 1`, `Type 2`, `HP`, `Attack`, `Defense`, `Sp. Atk`, `Sp. Def`, `Speed`, `Generation` e `Legendary`.

O arquivo `combats.csv` contém o histórico de batalhas utilizado nas análises. Cada registro representa um confronto entre dois Pokémon e informa o identificador do vencedor. Sua estrutura é composta pelos campos:

`First_pokemon`, `Second_pokemon` e `Winner`.

Por fim, o arquivo `tests.csv` contém pares de Pokémon destinados originalmente à aplicação de modelos de previsão de resultados. Diferentemente do arquivo de combates históricos, ele não contém a identificação do vencedor. Sua estrutura é composta por:

`First_pokemon` e `Second_pokemon`.

Arquivo	Conteúdo	Utilização no MVP
<code>pokemon.csv</code>	Cadastro e atributos dos Pokémon	Dimensão e análises
<code>combats.csv</code>	Histórico de batalhas e vencedores	Fato e métricas de desempenho
<code>tests.csv</code>	Pares de Pokémon sem resultado	Ingerido e tratado, mas não utilizado nas análises históricas

Fonte do conjunto de dados:

<https://www.kaggle.com/datasets/terminus7/pokemon-challenge>

2. Carga dos Dados

2.1. Ambiente de Desenvolvimento

O desenvolvimento do pipeline foi realizado no **Databricks Free Edition**, utilizado como ambiente cloud para armazenamento, processamento, transformação e análise dos dados.

A plataforma permitiu centralizar as diferentes etapas do projeto em um único ambiente, desde a ingestão dos arquivos CSV até a persistência das tabelas das camadas Bronze, Silver e Gold.

Para o processamento dos dados foram utilizados principalmente **Apache Spark e PySpark**, enquanto as consultas e análises da camada analítica foram desenvolvidas utilizando **Spark SQL**. As tabelas das diferentes camadas foram persistidas utilizando o formato **Delta**, permitindo que os dados processados fossem armazenados e posteriormente consumidos pelas etapas seguintes do pipeline.

As principais tecnologias utilizadas no projeto foram:

Tecnologia	Utilização
Databricks Free Edition	Ambiente cloud utilizado para desenvolvimento e execução do pipeline
Apache Spark	Motor de processamento distribuído dos dados
PySpark	Ingestão, transformação e validação dos dados
Spark SQL	Consultas e análises sobre as tabelas processadas
Delta Lake	Formato utilizado para persistência das tabelas das camadas do pipeline

2.2. Ingestão dos Arquivos

A primeira etapa do pipeline consistiu na disponibilização dos três arquivos CSV que compõem o conjunto de dados Pokémon Challenge no ambiente Databricks:

- `pokemon.csv`;
- `combats.csv`;
- `tests.csv`.

Os arquivos foram armazenados em um Volume no Databricks e utilizados como fonte para o processo de ingestão da camada Bronze.

A leitura dos arquivos foi realizada por meio do Spark, considerando a primeira linha dos arquivos como cabeçalho e permitindo a identificação inicial dos tipos de dados presentes em cada conjunto.

Após a leitura, os DataFrames resultantes foram inspecionados para verificar sua estrutura, quantidade de registros e schema antes da persistência na camada Bronze.

Essa etapa permitiu manter os arquivos originais como fonte do pipeline, separando fisicamente os dados de origem das tabelas posteriormente utilizadas nas etapas de transformação.

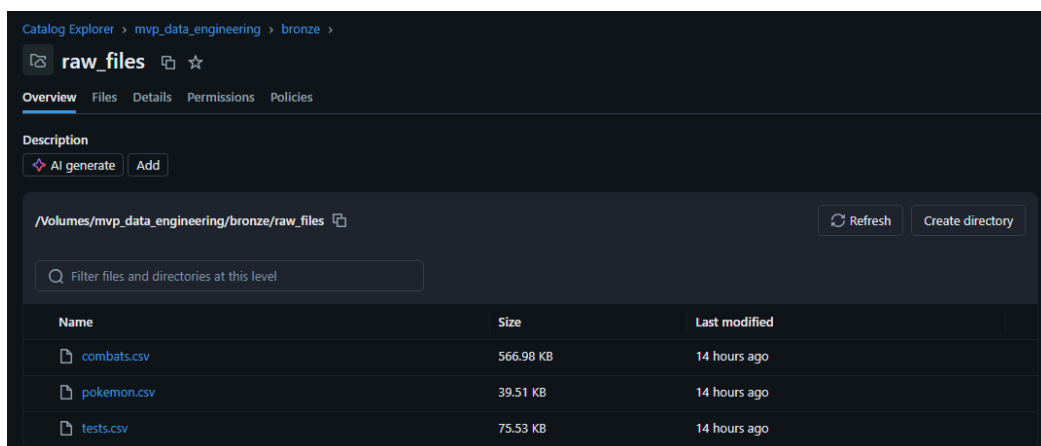


Figura 1 — Arquivos de origem armazenados no ambiente Databricks.

2.3. Construção da Camada Bronze

Após a ingestão dos arquivos de origem, os dados foram persistidos na **camada Bronze**, responsável por armazenar uma representação dos dados brutos com o mínimo possível de alterações.

Foram criadas três tabelas:

Arquivo de origem	Tabela Bronze
<code>pokemon.csv</code>	<code>bronze.pokemon_raw</code>
<code>combats.csv</code>	<code>bronze.combats_raw</code>
<code>tests.csv</code>	<code>bronze.tests_raw</code>

A tabela `pokemon_raw` preservou, por exemplo, os nomes originais das colunas do arquivo `pokemon.csv`, incluindo campos como `#`, `Name`, `Type 1`, `Type 2`, `Sp. Atk` e `Sp. Def`.

Essa decisão foi adotada para manter a camada Bronze próxima à estrutura recebida na origem. A padronização dos nomes das colunas, definição explícita dos tipos e demais transformações foram realizadas posteriormente na camada Silver.

Durante a persistência da tabela `pokemon_raw`, foi necessário considerar a presença de espaços e caracteres especiais nos nomes das colunas originais. Para possibilitar a preservação desse schema na tabela Delta, foi utilizada a configuração de **Column Mapping por nome**.

```
15 hours ago (14s) 10
#Persistir a camada bronze em Delta
(
  pokemon_bronze.write
    .format("delta")
    .option("delta.columnMapping.mode", "name")
    .option("delta.minReaderVersion", "2")
    .option("delta.minWriterVersion", "5")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.bronze.pokemon_raw")
)

(
  combats_bronze.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.bronze.combats_raw")
)

(
  tests_bronze.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.bronze.tests_raw")
)
> See performance \(3\) ✓ Checked for improvements
```

Figura 2 — Persistência da Bronze no ambiente Databricks.

As tabelas foram então persistidas no formato Delta dentro do schema `bronze`, passando a funcionar como ponto de partida para as demais etapas do pipeline.

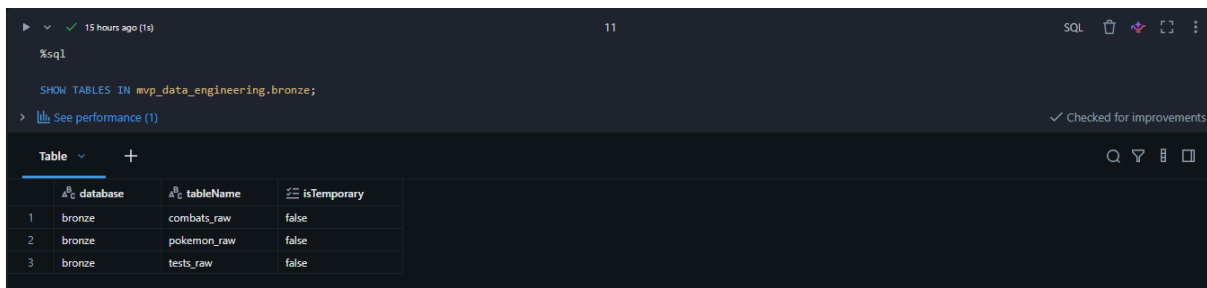


Figura 3 — Evidência da camada Bronze no ambiente Databricks.

2.4. Metadados de Ingestão

Além das colunas provenientes dos arquivos de origem, foram adicionados dois campos técnicos durante o processo de ingestão:

- `_ingestion_timestamp`;
- `_source_file`.

O campo `_ingestion_timestamp` registra o momento em que o dado foi processado pelo pipeline e inserido na camada Bronze.

Já `_source_file` identifica o arquivo responsável pela origem de cada registro, como `pokemon.csv`, `combats.csv` ou `tests.csv`.

Esses atributos não fazem parte originalmente do Pokémon Challenge e foram incluídos com a finalidade de proporcionar maior **rastreabilidade e linhagem dos dados** dentro do pipeline.

Dessa forma, torna-se possível identificar tanto a origem de um registro quanto o momento de sua ingestão.

Campo técnico	Descrição	Exemplo
<code>_ingestion_timestamp</code>	Data e horário de ingestão do registro	<code>2026-09-26 21:...</code>
<code>_source_file</code>	Arquivo de origem do registro	<code>pokemon.csv</code>

Os metadados foram preservados nas camadas Bronze e Silver para auxiliar na rastreabilidade do processo. Na camada Gold, esses campos foram removidos das tabelas analíticas por não representarem características necessárias às análises realizadas.

2.5. Evidências da Carga

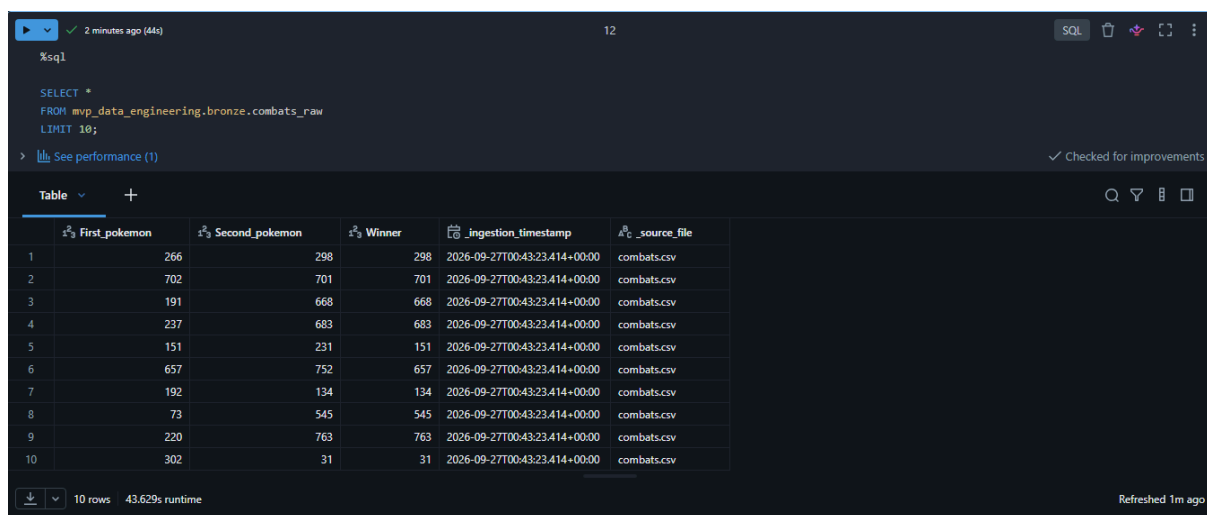
Após a persistência, foram realizadas consultas às tabelas da camada Bronze para confirmar que os dados haviam sido carregados corretamente.

A verificação permitiu confirmar a existência das três tabelas previstas:

- `pokemon_raw`;
- `combats_raw`;
- `tests_raw`.

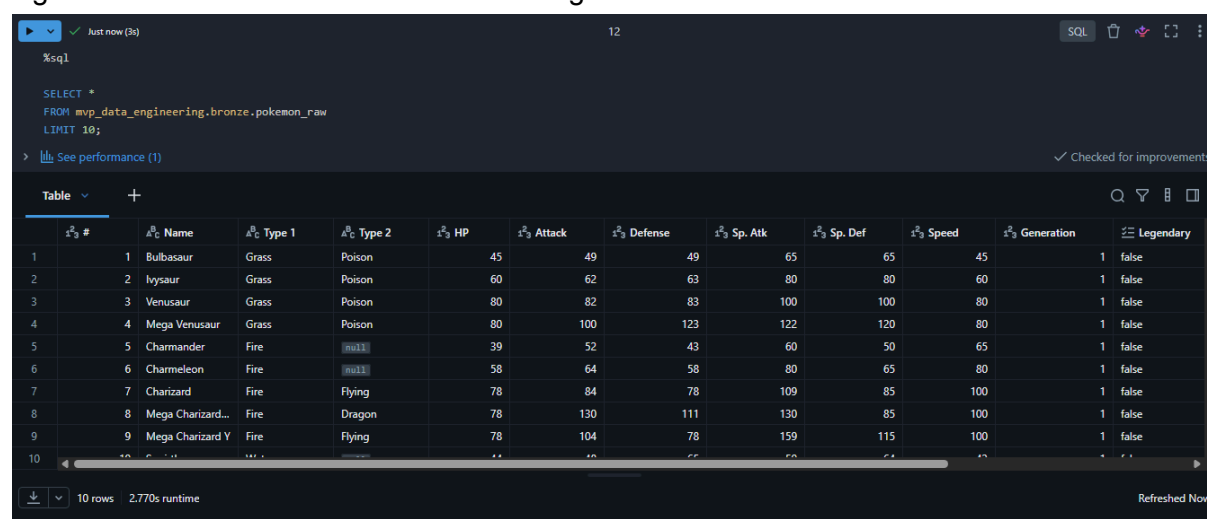
Também foram inspecionados registros das tabelas e seus respectivos schemas, verificando a preservação das colunas provenientes dos arquivos de origem e a presença dos metadados adicionados durante a ingestão.

Com a conclusão dessa etapa, os dados brutos passaram a estar disponíveis em tabelas Delta no ambiente Databricks, estabelecendo uma fonte persistente e controlada para as etapas posteriores de tratamento, validação e modelagem.



	First_pokemon	Second_pokemon	Winner	_ingestion_timestamp	_source_file
1	266	298	298	2026-09-27T00:43:23.414+00:00	combats.csv
2	702	701	701	2026-09-27T00:43:23.414+00:00	combats.csv
3	191	668	668	2026-09-27T00:43:23.414+00:00	combats.csv
4	237	683	683	2026-09-27T00:43:23.414+00:00	combats.csv
5	151	231	151	2026-09-27T00:43:23.414+00:00	combats.csv
6	657	752	657	2026-09-27T00:43:23.414+00:00	combats.csv
7	192	134	134	2026-09-27T00:43:23.414+00:00	combats.csv
8	73	545	545	2026-09-27T00:43:23.414+00:00	combats.csv
9	220	763	763	2026-09-27T00:43:23.414+00:00	combats.csv
10	302	31	31	2026-09-27T00:43:23.414+00:00	combats.csv

Figura 4 — Evidência dos Metadados da ingestão.



#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	false
2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	false
3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	false
4	Mega Venusaur	Grass	Poison	80	100	123	122	120	80	1	false
5	Charmander	Fire	null	39	52	43	60	50	65	1	false
6	Charmeleon	Fire	null	58	64	58	80	65	80	1	false
7	Charizard	Fire	Flying	78	84	78	109	85	100	1	false
8	Mega Charizard...	Fire	Dragon	78	130	111	130	85	100	1	false
9	Mega Charizard Y	Fire	Flying	78	104	78	159	115	100	1	false
10											

Figura 5 — Evidência da materialização das tabelas no ambiente Databricks.

A partir das tabelas da camada Bronze, o pipeline prossegue para a camada Silver, na qual são realizadas as transformações de padronização, tipagem e validação da qualidade dos dados.

3. Modelagem e Catálogo de Dados

3.1. Arquitetura Medallion

A arquitetura do projeto foi estruturada seguindo o modelo **Medallion**, no qual os dados passam progressivamente por diferentes níveis de tratamento e organização. O pipeline foi dividido nas camadas Bronze, Silver e Gold, cada uma com uma responsabilidade específica.

A **camada Bronze** representa o primeiro nível de persistência dos dados. Nela são armazenadas as informações provenientes dos arquivos de origem, preservando sua estrutura original e adicionando apenas metadados técnicos relacionados à ingestão.

A **camada Silver** contém os dados padronizados e validados. Nessa etapa são realizadas transformações como padronização dos nomes das colunas, definição explícita dos tipos de dados, validação de campos obrigatórios, verificação de integridade referencial e aplicação de regras de negócio.

Por fim, a **camada Gold** disponibiliza os dados preparados para consumo analítico. Nessa camada foi construída uma modelagem dimensional simplificada que permite realizar as análises propostas sem necessidade de acessar diretamente as estruturas intermediárias da camada Silver.

A separação entre as camadas permite organizar as responsabilidades do pipeline, preservar os dados de origem e disponibilizar estruturas específicas para diferentes finalidades de consumo.

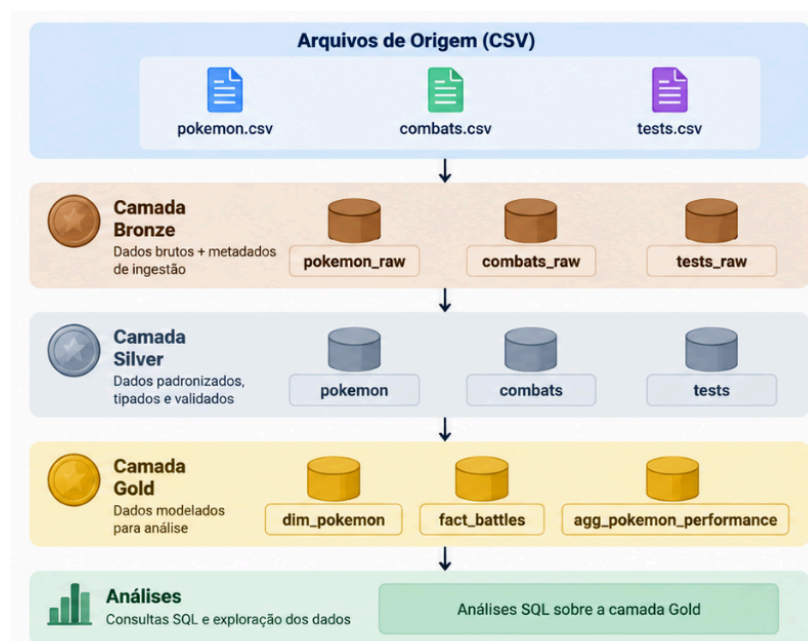


Figura 6 — Arquitetura Medalhão, feita na geração de imagem do ChatGPT.

3.2. Modelo de Dados da Camada Gold

A camada Gold foi desenvolvida com foco nas perguntas analíticas estabelecidas no início do projeto. Para isso, foi utilizada uma **modelagem dimensional simplificada**, composta por uma dimensão, uma tabela fato e uma tabela agregada.

As três principais estruturas são:

- **dim_pokemon**;
- **fact_battles**;
- **agg_pokemon_performance**.

A tabela **dim_pokemon** concentra os atributos descritivos de cada Pokémon, como nome, tipos, atributos de batalha, geração e classificação como Legendary.

A tabela **fact_battles** representa os eventos de batalha registrados no conjunto de dados. Cada registro corresponde a uma batalha e contém os identificadores dos dois participantes, do vencedor e do perdedor.

A dimensão Pokémon assume diferentes papéis em relação à tabela fato, uma vez que um mesmo **pokemon_id** pode representar o primeiro participante, o segundo participante, o vencedor ou o perdedor de uma batalha.

Por fim, a tabela **agg_pokemon_performance** contém métricas agregadas de desempenho por Pokémon, disponibilizando a quantidade total de batalhas, vitórias, derrotas e a respectiva taxa de vitória.

Essa tabela foi criada para simplificar consultas analíticas recorrentes e evitar que essas métricas precisem ser recalculadas diretamente a partir de todos os registros da tabela fato a cada nova análise.

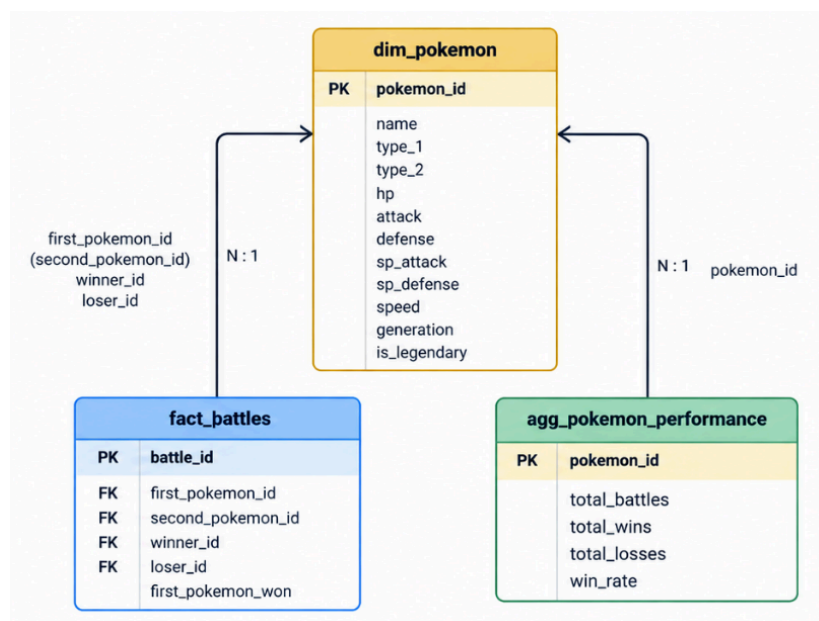


Figura 7 — Modelo lógico da camada Gold, feita na geração de imagem do ChatGPT.

3.3. Relacionamentos e Granularidade

A granularidade de cada tabela foi definida de acordo com sua finalidade dentro do modelo analítico.

A tabela `dim_pokemon` possui granularidade de **um registro por Pokémon**, utilizando `pokemon_id` como identificador lógico único.

A tabela `fact_battles` possui granularidade de **um registro por batalha**, utilizando `battle_id` como identificador da ocorrência. Os campos `first_pokemon_id`, `second_pokemon_id`, `winner_id` e `loser_id` estabelecem relacionamentos lógicos com `dim_pokemon.pokemon_id`.

A tabela `agg_pokemon_performance`, por sua vez, possui granularidade de **um registro por Pokémon** e se relaciona à dimensão por meio de `pokemon_id`.

A estrutura dos relacionamentos pode ser resumida da seguinte forma:

Tabela	Granularidade	Identificador	Relacionamentos
<code>dim_pokemon</code>	1 registro por Pokémon	<code>pokemon_id</code>	Entidade central de Pokémon
<code>fact_battles</code>	1 registro por batalha	<code>battle_id</code>	IDs dos participantes, vencedor e perdedor → <code>dim_pokemon.pokemon_id</code>
<code>agg_pokemon_performance</code>	1 registro por Pokémon	<code>pokemon_id</code>	<code>pokemon_id</code> → <code>dim_pokemon.pokemon_id</code>

Os relacionamentos representam chaves primárias e estrangeiras **lógicas**, utilizadas para garantir a consistência das consultas e das transformações do pipeline.

3.4. Catálogo de Dados — **dim_pokemon**

A tabela **dim_pokemon** disponibiliza os atributos cadastrais e características utilizadas para descrever cada Pokémon nas análises.

Campo	Tipo	Descrição	Domínio / Regra	Origem / Transformação
pokemon_id	INT	Identificador único do Pokémon	Valor inteiro positivo e único	silver.pokemon.pokemon_id
name	STRING	Nome do Pokémon	Não nulo	silver.pokemon.name
type_1	STRING	Tipo primário do Pokémon	Categoria válida de tipo	silver.pokemon.type_1
type_2	STRING	Tipo secundário do Pokémon	Categoria válida ou nulo	silver.pokemon.type_2
hp	INT	Pontos de vida	Valor inteiro não negativo	silver.pokemon.hp
attack	INT	Atributo de ataque físico	Valor inteiro não negativo	silver.pokemon.attack
defense	INT	Atributo de defesa física	Valor inteiro não negativo	silver.pokemon.defense
sp_attack	INT	Atributo de ataque especial	Valor inteiro não negativo	silver.pokemon.sp_attack
sp_defense	INT	Atributo de defesa especial	Valor inteiro não negativo	silver.pokemon.sp_defense
speed	INT	Atributo de velocidade	Valor inteiro não negativo	silver.pokemon.speed
generation	INT	Geração à qual o Pokémon pertence	Valor pertencente às gerações presentes na fonte	silver.pokemon.generation
is_legendary	BOOLEAN	Indica se o Pokémon é classificado como Legendary	true ou false	silver.pokemon.is_legendary

Os campos desta dimensão têm origem no arquivo `pokemon.csv`, sendo inicialmente persistidos em `bronze.pokemon_raw`, posteriormente padronizados e validados em `silver.pokemon` e, finalmente, disponibilizados na dimensão Gold.

3.5. Catálogo de Dados — `fact_battles`

A tabela `fact_battles` representa os eventos históricos de batalha presentes no conjunto de dados.

Campo	Tipo	Descrição	Domínio / Regra	Origem / Transformação
<code>battle_id</code>	BIGINT	Identificador da batalha	Identificador técnico da ocorrência	Criado durante o tratamento dos registros de <code>silver.combats</code>
<code>first_pokemon_id</code>	INT	Identificador do primeiro participante	Deve existir em <code>dim_pokemon.pokemon_id</code>	<code>silver.combats.first_pokemon_id</code>
<code>second_pokemon_id</code>	INT	Identificador do segundo participante	Deve existir em <code>dim_pokemon.pokemon_id</code>	<code>silver.combats.second_pokemon_id</code>
<code>winner_id</code>	INT	Identificador do vencedor	Deve corresponder a um dos participantes e existir em <code>dim_pokemon</code>	<code>silver.combats.winner_id</code>
<code>loser_id</code>	INT	Identificador do Pokémon derrotado	Deve corresponder ao participante diferente de <code>winner_id</code>	Derivado de <code>first_pokemon_id</code> , <code>second_pokemon_id</code> e <code>winner_id</code>
<code>first_pokemon_won</code>	BOOLEAN	Indica se o primeiro Pokémon venceu a batalha	<code>true</code> ou <code>false</code>	Comparação entre <code>winner_id</code> e <code>first_pokemon_id</code>

A utilização dos identificadores de vencedor e perdedor facilita consultas que comparam diretamente os atributos dos dois grupos, enquanto `first_pokemon_won` permite identificar rapidamente o resultado do primeiro participante da batalha.

3.6. Catálogo de Dados — **agg_pokemon_performance**

A tabela **agg_pokemon_performance** disponibiliza métricas consolidadas de desempenho por Pokémon.

Para sua construção, cada registro de **fact_battles** foi transformado conceitualmente em duas participações: uma referente ao primeiro Pokémon e outra referente ao segundo. Cada participação recebeu um indicador de vitória de acordo com o resultado da batalha.

As participações foram posteriormente agregadas por **pokemon_id**, permitindo calcular as métricas de desempenho.

Campo	Tipo	Descrição	Domínio / Regra	Origem / Transformação
pokemon_id	INT	Identificador do Pokémon	Deve existir em dim_pokemon.pokemon_id	Derivado das participações de fact_battles
total_battles	BIGINT	Quantidade total de batalhas disputadas	Valor inteiro ≥ 0	Contagem das participações do Pokémon
total_wins	BIGINT	Quantidade total de vitórias	Valor inteiro ≥ 0	Soma das participações classificadas como vitória
total_losses	BIGINT	Quantidade total de derrotas	total_battles - total_wins	Campo derivado
win_rate	DOUBLE	Proporção de vitórias do Pokémon	Entre 0 e 1 quando existem batalhas; nulo quando não há participações	total_wins / total_battles

Para Pokémon sem registros históricos de batalha, as quantidades de batalhas, vitórias e derrotas são representadas por zero, enquanto a taxa de vitória permanece nula. Essa decisão permite diferenciar um Pokémon que participou de batalhas e não obteve vitórias de outro para o qual não existem observações suficientes para calcular a métrica.

3.7. Evidências da Modelagem

Após a construção das estruturas analíticas, as três tabelas foram persistidas no schema **gold** no formato Delta:

- `dim_pokemon;`
- `fact_battles;`
- `agg_pokemon_performance.`

A persistência dessas tabelas representa a etapa final de transformação do pipeline antes do consumo analítico. As consultas realizadas posteriormente utilizam exclusivamente as estruturas disponibilizadas na camada Gold.

```

(
  dim_pokemon.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.gold.dim_pokemon")
)

(
  fact_battles.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.gold.fact_battles")
)

```

> [See performance \(2\)](#) ✓ Checked for improvements

```

%sql

SHOW TABLES IN mvp_data_engineering.gold;

```

> [See performance \(1\)](#) ✓ Checked for improvements

	database	tableName	isTemporary
1	gold	dim_pokemon	false
2	gold	fact_battles	false

Figura 8 — Persistência da Gold no ambiente Databricks.

```

(
  agg_pokemon_performance.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable(
      "mvp_data_engineering.gold.agg_pokemon_performance"
    )
)

```

> [See performance \(1\)](#) ✓ Checked for improvements

```

%sql

SHOW TABLES IN mvp_data_engineering.gold;

```

> [See performance \(1\)](#) ✓ Checked for improvements

	database	tableName	isTemporary
1	gold	agg_pokemon_performan...	false
2	gold	dim_pokemon	false
3	gold	fact_battles	false

Figura 9 — Persistência da Gold com a tabela analítica no ambiente Databricks.

A inspeção das tabelas permitiu verificar tanto a estrutura dos dados quanto as transformações realizadas durante a modelagem.

The screenshot shows a Databricks SQL interface. At the top, a status bar indicates '15 hours ago (3s)' and '36'. Below it, a SQL query is displayed in a dark-themed editor. The query selects various attributes from a table named 'agg_pokemon_performance' and joins it with a table named 'dim_pokemon'. The query is ordered by 'win_rate' in descending order and limited to 10 rows. Below the query editor, a table view shows the results of the query. The table has 8 columns: 'name', 'type_1', 'speed', 'total_battles', 'total_wins', 'total_losses', 'win_rate_pct', and an additional column labeled '1.2 win_rate_pct'. The results are displayed as a list of 10 rows, each representing a different Pokémon.

```

%sql

SELECT
  p.name,
  p.type_1,
  p.speed,
  a.total_battles,
  a.total_wins,
  a.total_losses,
  ROUND(a.win_rate * 100, 2) AS win_rate_pct
FROM mvp_data_engineering.gold.agg_pokemon_performance a
INNER JOIN mvp_data_engineering.gold.dim_pokemon p
  ON a.pokemon_id = p.pokemon_id
WHERE a.total_battles >= 20
ORDER BY a.win_rate DESC
LIMIT 10;

```

	name	type_1	speed	total_battles	total_wins	total_losses	win_rate_pct	1.2 win_rate_pct
1	Mega Aerodactyl	Rock	150	129	127	2	98.45	
2	Weavile	Dark	125	119	116	3	97.48	
3	Tornadus Therian Forme	Flying	121	125	121	4	96.8	
4	Mega Beedrill	Bug	145	119	115	4	96.64	
5	Aerodactyl	Rock	130	141	136	5	96.45	
6	Mega Lopunny	Normal	135	129	124	5	96.12	
7	Greninja	Water	122	127	122	5	96.06	
8	Meloetta Pirouette For...	Normal	128	123	118	5	95.93	
9	Mega Mewtwo Y	Psychic	140	125	119	6	95.2	
10	Mega Sharpedo	Water	105	120	114	6	95	

Figura 10 — Exemplo de consulta à Gold no ambiente Databricks.

4. Pipeline de Dados

Nesta seção, vamos documentar a implementação do pipeline, explicando como os dados percorrem as camadas Bronze, Silver e Gold, quais transformações são realizadas e como os notebooks estão organizados.

A diferença em relação às seções anteriores é que, enquanto a seção 2 documentou a carga e a seção 3 apresentou a arquitetura e a modelagem, a seção 4 demonstra como o pipeline foi implementado e executado.

4.1. Visão Geral do Pipeline

O pipeline foi desenvolvido no ambiente **Databricks**, utilizando **PySpark** e **Spark SQL** para implementar as etapas de ingestão, transformação, validação e disponibilização dos dados.

O fluxo de processamento foi dividido em quatro etapas principais:

1. Ingestão: leitura dos arquivos CSV e persistência dos dados brutos na camada Bronze, acompanhados de metadados técnicos.
2. Tratamento: leitura das tabelas Bronze, padronização dos campos, definição dos tipos de dados e aplicação de regras de qualidade para construção da camada Silver.
3. Modelagem: transformação dos dados da Silver em estruturas dimensionais e agregadas, persistidas na camada Gold.
4. Análise: execução de consultas SQL sobre as tabelas Gold para responder às perguntas de negócio estabelecidas.

Cada etapa foi implementada em um notebook específico, permitindo organizar o código de acordo com sua responsabilidade e facilitar a identificação das transformações realizadas.

A execução dos notebooks respeita a sequência das dependências entre as camadas, garantindo que cada etapa utilize os dados persistidos pela etapa anterior.

4.2. Pipeline Source → Bronze

A primeira etapa do pipeline foi implementada no notebook `01_ingestion_bronze`, responsável pela leitura dos arquivos de origem e pela construção da camada Bronze.

Inicialmente, foram criados o catálogo `mvp_data_engineering` e os schemas correspondentes às camadas Bronze, Silver e Gold, permitindo organizar as tabelas em diferentes níveis de processamento.

Em seguida, os arquivos `pokemon.csv`, `combats.csv` e `tests.csv`, previamente disponibilizados em um Volume do Databricks, foram carregados em DataFrames por meio do PySpark. Durante a ingestão, foram adicionados os campos técnicos `_ingestion_timestamp` e `_source_file`, responsáveis por registrar o momento do processamento e a origem dos registros. A persistência foi realizada no formato Delta, resultando nas tabelas `bronze.pokemon_raw`, `bronze.combats_raw` e `bronze.tests_raw`.

Ao final dessa etapa, os dados de origem estavam disponíveis em tabelas persistidas, possibilitando seu consumo pelas etapas seguintes sem a necessidade de realizar novamente a leitura dos arquivos CSV. **As evidências foram postas na seção 2.5 deste documento.**

4.3. Pipeline Bronze → Silver

A segunda etapa foi desenvolvida no notebook `02_bronze_to_silver`, responsável pelo tratamento e pela padronização dos dados provenientes da camada Bronze.

Inicialmente, foram realizadas a leitura das três tabelas Bronze e a transformação dos nomes das colunas para o padrão `snake_case`, facilitando sua manipulação nas etapas posteriores.

Também foram definidos explicitamente os tipos de dados, convertendo os atributos numéricos, identificadores e indicadores booleanos para os formatos apropriados.

```
▶ 16 hours ago (1s) 5 Python

#Padronizando os nomes das colunas e tipagem explícita
from pyspark.sql.functions import col

pokemon_silver = pokemon_bronze.select(
    col("`#`").cast("int").alias("pokemon_id"),
    col("`Name`").cast("string").alias("name"),
    col("`Type 1`").cast("string").alias("type_1"),
    col("`Type 2`").cast("string").alias("type_2"),
    col("`HP`").cast("int").alias("hp"),
    col("`Attack`").cast("int").alias("attack"),
    col("`Defense`").cast("int").alias("defense"),
    col("`Sp. Atk`").cast("int").alias("sp_attack"),
    col("`Sp. Def`").cast("int").alias("sp_defense"),
    col("`Speed`").cast("int").alias("speed"),
    col("`Generation`").cast("int").alias("generation"),
    col("`Legendary`").cast("boolean").alias("is_legendary"),
    col("_ingestion_timestamp"),
    col("_source_file")
)
```

Figura 11 — Padronização da tabela na Silver no ambiente Databricks.

Na entidade Pokémon, foram executadas verificações relacionadas à presença de valores nulos em campos obrigatórios, unicidade dos identificadores e consistência dos atributos. O campo `type_2` foi tratado como opcional, uma vez que determinados Pokémon possuem apenas um tipo.

```
▶ 16 hours ago (<1s) 7 Python

#verificação de invalidez dos pokémon listados
invalid_pokemon = pokemon_silver.filter(
    col("pokemon_id").isNull()
    | col("type_1").isNull()
    | col("hp").isNull()
    | col("attack").isNull()
    | col("defense").isNull()
    | col("sp_attack").isNull()
    | col("sp_defense").isNull()
    | col("speed").isNull()
    | col("generation").isNull()
    | col("is_legendary").isNull()
    | (col("hp") < 0)
    | (col("attack") < 0)
    | (col("defense") < 0)
    | (col("sp_attack") < 0)
    | (col("sp_defense") < 0)
    | (col("speed") < 0)
)

> invalid_pokemon: pyspark.sql.connect.dataframe.DataFrame = [pokemon_id: integer,
name: string ... 12 more fields]

▶ 16 hours ago (1s) 8

print(
    "Pokémon considerados inválidos:",
    invalid_pokemon.count()
)

> See performance (1) ✓ Checked for improvements

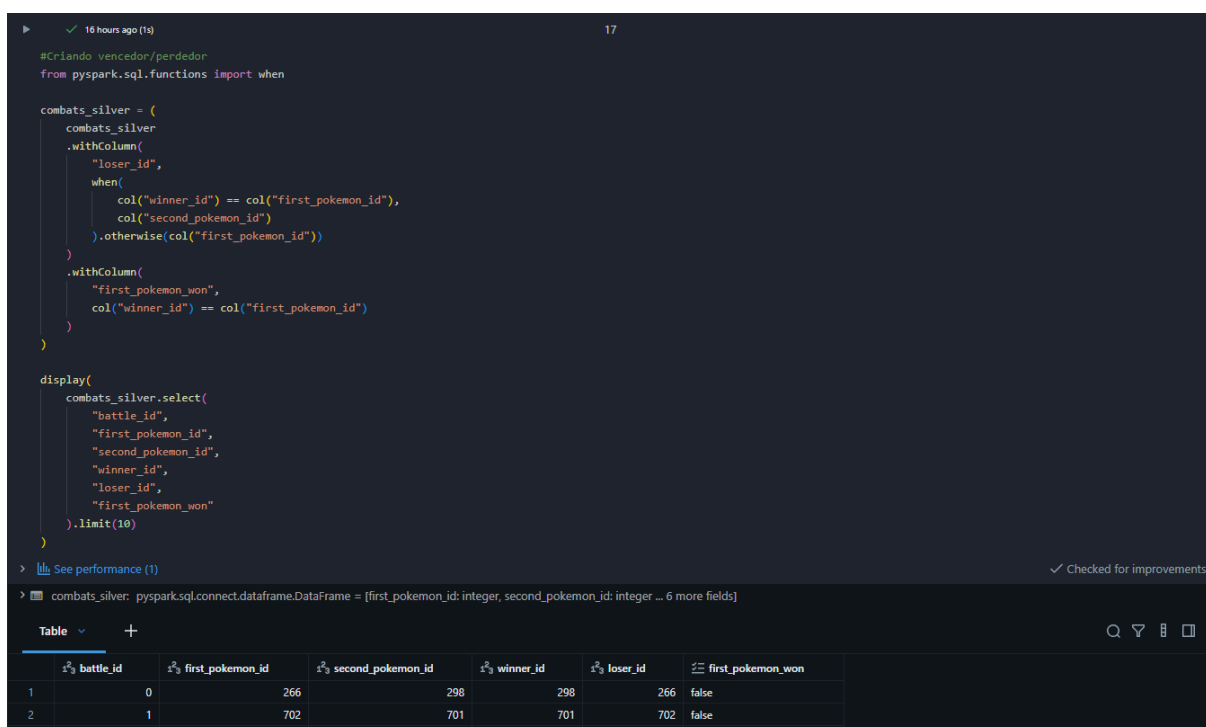
Pokémon considerados inválidos: 0
```

Figura 12 — Validação de dados da tabela na Silver no ambiente Databricks.

Para os dados de combates, foram aplicadas regras de validação destinadas a verificar se os identificadores dos participantes e vencedores eram válidos e se o vencedor de cada batalha correspondia a um dos participantes.

Além dessas verificações, foram criados atributos derivados que simplificam a modelagem posterior:

- **battle_id**: identificador técnico de cada registro de batalha;
- **loser_id**: identificador do Pokémon derrotado;
- **first_pokemon_won**: indicador booleano que identifica se o primeiro participante venceu.



```
#Criando vencedor/perdedor
from pyspark.sql.functions import when

combats_silver = (
    combats_silver
    .withColumn(
        "loser_id",
        when(
            col("winner_id") == col("first_pokemon_id"),
            col("second_pokemon_id")
        ).otherwise(col("first_pokemon_id"))
    )
    .withColumn(
        "first_pokemon_won",
        col("winner_id") == col("first_pokemon_id")
    )
)

display(
    combats_silver.select(
        "battle_id",
        "first_pokemon_id",
        "second_pokemon_id",
        "winner_id",
        "loser_id",
        "first_pokemon_won"
    ).limit(10)
)
```

See performance (1) ✓ Checked for improvements

combats_silver: pyspark.sql.connect.dataframe.DataFrame = [first_pokemon_id: integer, second_pokemon_id: integer ... 6 more fields]

	battle_id	first_pokemon_id	second_pokemon_id	winner_id	loser_id	first_pokemon_won
1	0	266	298	298	266	false
2	1	702	701	701	702	false

Figura 13 — Criação das colunas de identificação na Silver no ambiente Databricks.

O arquivo **tests.csv** também foi tratado, recebendo nomes de colunas padronizados e um identificador técnico denominado **test_id**.

```
▶ 16 hours ago (4s) 23 Python

tests_silver = tests_bronze.select(
    col("First_pokemon").cast("int").alias("first_pokemon_id"),
    col("Second_pokemon").cast("int").alias("second_pokemon_id"),
    col("_ingestion_timestamp"),
    col("_source_file")
)

tests_silver = tests_silver.withColumn(
    "test_id",
    monotonically_increasing_id()
)

display(tests_silver.limit(10))

> See performance \(1\) ✓ Checked for improvements

> tests_silver: pyspark.sql.connect.dataframe.DataFrame = [first_pokemon_id: integer,
second_pokemon_id: integer ... 3 more fields]

Table + 🔍 ⚙️ □

1 129 117 2026-09-27T00:43:27.335+00:00
2 660 211 2026-09-27T00:43:27.335+00:00
```

Figura 14 — Padronização da tabela no ambiente Databricks.

Após as transformações e validações, os resultados foram persistidos nas tabelas `silver.pokemon`, `silver.combats` e `silver.tests`.

▶ ✓ 16 hours ago (10s) 25

```

(
  pokemon_silver.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.silver.pokemon")
)

(
  combats_silver.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.silver.combats")
)

(
  tests_silver.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("mvp_data_engineering.silver.tests")
)

```

> [See performance \(3\)](#) ✓ Checked for improvements

+ Code + Text Genie Code

▶ ✓ 16 hours ago (1s) 26

```

%sql

SHOW TABLES IN mvp_data_engineering.silver;

```

> [See performance \(1\)](#) ✓ Checked for improvements

Table ▾ + 🔍 ⚙️ 📄

	database	tableName	isTemporary
1	silver	combats	false
2	silver	pokemon	false
3	silver	tests	false

Figura 15 — Persistência das tabelas na camada Silver no ambiente Databricks.

A camada Silver passou, assim, a disponibilizar uma representação padronizada dos dados de origem, preservando os registros que atenderam às regras estabelecidas e preparando-os para a modelagem analítica.

4.4. Pipeline Silver → Gold

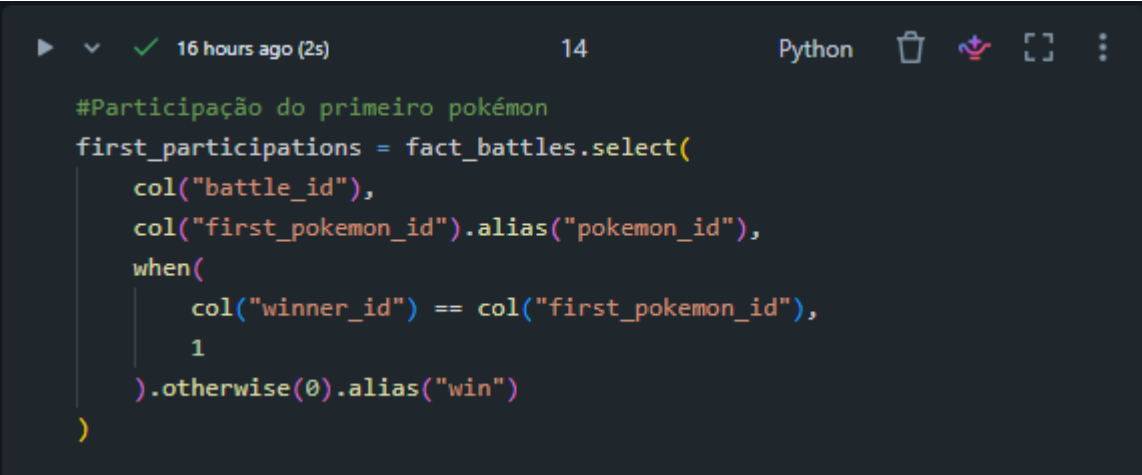
A terceira etapa foi implementada no notebook `03_silver_to_gold`, responsável pela construção das estruturas analíticas utilizadas para responder às perguntas de negócio.

A partir da tabela `silver.pokemon`, foi construída a dimensão `gold.dim_pokemon`, contendo os atributos descritivos e estatísticos de cada Pokémon.

Em seguida, os registros de `silver.combats` foram utilizados para construir `gold.fact_battles`. Essa tabela representa as ocorrências de batalhas, preservando os identificadores dos participantes e disponibilizando os atributos derivados relacionados ao resultado de cada confronto.

Por fim, foi desenvolvida a tabela `gold.agg_pokemon_performance`, responsável por consolidar as métricas de desempenho individuais.

Para sua construção, cada batalha foi desmembrada em duas participações: uma correspondente ao primeiro Pokémon e outra ao segundo. Cada participação recebeu um indicador de vitória, posteriormente utilizado para calcular as quantidades totais de batalhas, vitórias e derrotas.



```
▶ 16 hours ago (2s) 14 Python
```

```
#Participação do primeiro pokémon
first_participations = fact_battles.select(
    col("battle_id"),
    col("first_pokemon_id").alias("pokemon_id"),
    when(
        col("winner_id") == col("first_pokemon_id"),
        1
    ).otherwise(0).alias("win")
)
```

Figura 16 — Criação da variável do primeiro pokémon na Gold no ambiente Databricks.

```

▶  ✓ 16 hours ago (<1s) 15 Python
#Participação do segundo pokémon
second_participations = fact_battles.select(
    col("battle_id"),
    col("second_pokemon_id").alias("pokemon_id"),
    when(
        col("winner_id") == col("second_pokemon_id"),
        1
    ).otherwise(0).alias("win")
)

```

Figura 17 — Criação da variável do segundo pokémon na Gold no ambiente Databricks.

A taxa de vitória foi calculada pela divisão entre o total de vitórias e o total de batalhas disputadas.

```

▶  ✓ 16 hours ago (<1s) 22 Python
#Criando taxa de vitória
pokemon_performance = (
    pokemon_performance
    .withColumn(
        "win_rate",
        round(
            col("total_wins") / col("total_battles"),
            4
        )
    )
)

```

Figura 18 — Criação da taxa de vitória na Gold no ambiente Databricks.

16 hours ago (1s) 23

```
display(
  pokemon_performance
    .orderBy(col("win_rate").desc())
    .limit(10)
)
```

> [See performance \(1\)](#)

Table +

	¹ ₃ pokemon_id	¹ ₃ total_battles	¹ ₃ total_wins	¹ ₃ total_losses	1.2 win_rate
1	155	129	127	2	0.9845
2	513	119	116	3	0.9748
3	704	125	121	4	0.968
4	20	119	115	4	0.9664
5	154	141	136	5	0.9645
6	477	129	124	5	0.9612
7	727	127	122	5	0.9606
8	717	123	118	5	0.9593
9	165	125	119	6	0.952
10	350	120	114	6	0.95

Figura 19 — Evidência dos cálculos para criação da tabela na Gold no ambiente Databricks

A agregação foi relacionada à dimensão Pokémon para garantir que todos os Pokémon cadastrados fossem representados, inclusive aqueles sem registros de participação em batalhas.

Para esses casos, as quantidades de batalhas, vitórias e derrotas foram preenchidas com zero, enquanto a taxa de vitória permaneceu nula, evitando atribuir uma taxa de desempenho a Pokémon sem observações históricas.

Após a construção das três estruturas, foram realizadas validações de consistência, incluindo a igualdade entre batalhas e a soma de vitórias e derrotas, os limites permitidos para a taxa de vitória e a correspondência entre o número de batalhas e o total de vitórias registrado.

As tabelas resultantes foram persistidas no schema Gold, tornando-se a fonte das consultas analíticas realizadas na etapa seguinte.

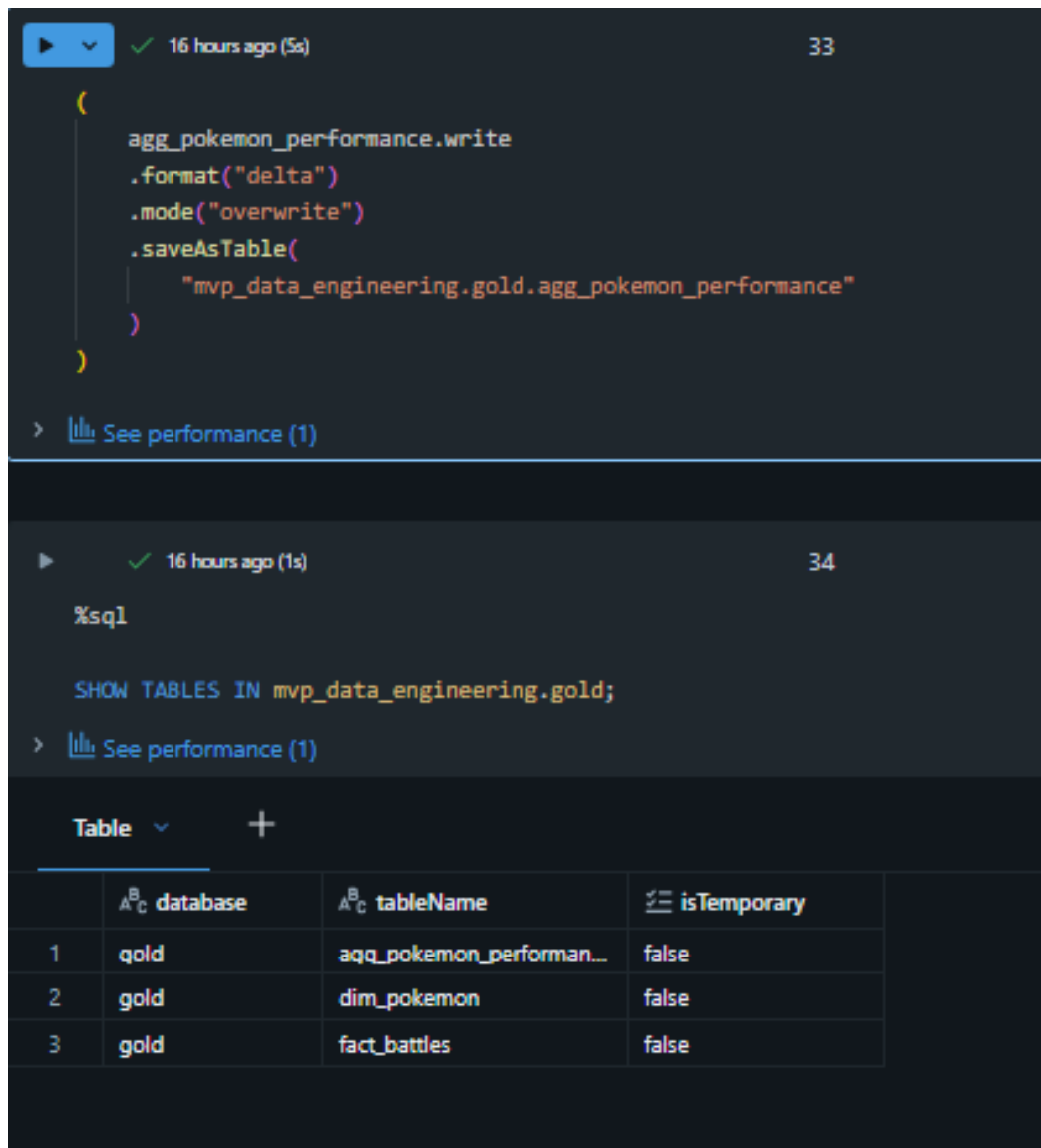


Figura 20 — Persistência das tabelas na Gold no ambiente Databricks

4.5. Organização dos Notebooks e Scripts

Para facilitar a organização e a reprodução do pipeline, o desenvolvimento foi dividido em quatro notebooks, cada um responsável por uma etapa específica do processamento.

Notebook	Responsabilidade
01_ingestion_bronze	Leitura dos CSVs, inclusão de metadados e persistência na Bronze

02_bronze_to_silver	Padronização, tipagem, validações e construção da Silver
03_silver_to_gold	Modelagem dimensional, agregação de métricas e construção da Gold
04_data_analysis	Consultas SQL e análises das perguntas de negócio

Essa divisão permite executar e verificar as etapas individualmente, além de facilitar a identificação de possíveis problemas em cada fase do processamento. Os notebooks serão disponibilizados no repositório GitHub do projeto, acompanhados da documentação necessária para compreender sua finalidade e a ordem de execução.

5. Qualidade dos Dados

5.1. Estratégia de Qualidade

A qualidade dos dados foi avaliada ao longo do pipeline com o objetivo de garantir que as informações disponibilizadas para consumo analítico fossem consistentes com as regras estabelecidas para o projeto. As verificações foram concentradas principalmente durante a transformação da camada Bronze para a Silver e complementadas por validações sobre as estruturas produzidas na camada Gold. Foram considerados cinco critérios principais de qualidade:

- **Completeness:** Verificação da presença de valores nos campos considerados obrigatórios
- **Consistência:** Validação dos valores e das regras de negócio associadas aos registros
- **Unicidade:** Verificação de duplicidades em identificadores e registros
- **Integridade:** Verificação da coerência entre identificadores e relacionamentos das entidades
- **Outliers:** Análise das distribuições e dos valores dos atributos numéricos

As regras foram definidas considerando tanto a estrutura dos dados quanto seu significado dentro do contexto das batalhas Pokémon.

A abordagem adotada buscou evitar tratamentos desnecessários. Dessa forma, registros não foram removidos ou alterados apenas por apresentarem características menos frequentes, sendo realizadas correções somente quando justificadas pelas regras definidas para cada entidade.

5.2. Qualidade da Entidade Pokémon

Para os registros de Pokémon, foram avaliadas inicialmente a completude e a unicidade dos principais atributos. O campo **pokemon_id** foi considerado obrigatório e utilizado como identificador lógico da entidade. Foram realizadas verificações para identificar valores nulos, identificadores duplicados ou valores incompatíveis com o domínio esperado.

Os campos **name** e **type_1** também foram considerados obrigatórios. Em contrapartida, valores nulos em **type_2** foram considerados válidos, pois determinados Pokémon possuem apenas um tipo. Dessa forma, a ausência desse atributo não foi classificada como problema de qualidade.

Os atributos numéricos de batalha — **hp**, **attack**, **defense**, **sp_attack**, **sp_defense** e **speed** — foram analisados quanto à presença de valores nulos, tipagem e ocorrência de valores negativos.

Também foram analisadas suas distribuições por meio de estatísticas descritivas, permitindo identificar valores extremos e avaliar possíveis outliers. A identificação de um valor estatisticamente elevado, entretanto, não foi utilizada isoladamente como justificativa para exclusão. Considerando que diferentes Pokémon podem possuir atributos naturalmente distintos, os valores foram preservados quando não representam violações das regras estabelecidas.

As principais regras aplicadas à entidade Pokémon podem ser resumidas da seguinte forma:

Validação	Regra esperada
Identificador nulo	pokemon_id não pode ser nulo
Unicidade	pokemon_id deve ser único
Identificador válido	pokemon_id > 0
Nome	name não pode ser nulo
Tipo primário	type_1 não pode ser nulo

Tipo secundário	Pode assumir valor nulo
Atributos de batalha	Não podem ser nulos ou negativos
Geração	Deve pertencer ao domínio presente no conjunto de dados
Legendary	Deve ser representado como valor booleano

5.3. Qualidade dos Combates

Os registros de batalha exigem validações adicionais devido à existência de relacionamentos entre diferentes identificadores de Pokémon. Uma das principais regras verificadas determina que o vencedor de uma batalha deve necessariamente corresponder a um dos dois Pokémon participantes. Assim, para cada registro, foi validada a seguinte condição:

winner_id = first_pokemon_id ou winner_id = second_pokemon_id.

A análise realizada não identificou registros nos quais o vencedor estivesse fora dos participantes do confronto.

Também foram investigadas ocorrências de batalhas nas quais **first_pokemon_id** e **second_pokemon_id** fossem iguais. Essas ocorrências foram tratadas como uma condição a ser investigada, em vez de serem automaticamente removidas, uma vez que a simples identificação de uma possível anomalia não seria suficiente para determinar que o registro estivesse incorreto. Porém, não foram identificadas batalhas de uma mesma “espécie” de Pokémon.

```

> 20 hours ago (1s) 19
#Verificando se o vencedor está na batalha
invalid_winners = combats_silver.filter(
    (col("winner_id") != col("first_pokemon_id"))
    & (col("winner_id") != col("second_pokemon_id"))
)

print(
    "Vencedores inconsistentes:",
    invalid_winners.count()
)

> See performance (1) ✓ Checked for improvements
> invalid_winners: pyspark.sql.connect.dataframe.DataFrame = [first_pokemon_id: integer, second_pokemon_id: integer ... 6 more fields]
Vencedores inconsistentes: 0

> 20 hours ago (1s) 20
#Verificando se há batalhas entre o mesmo Pokémon
self_battles = combats_silver.filter(
    col("first_pokemon_id")
    == col("second_pokemon_id")
)

print(
    "Self battles:",
    self_battles.count()
)

> See performance (1) ✓ Checked for improvements
> self_battles: pyspark.sql.connect.dataframe.DataFrame = [first_pokemon_id: integer, second_pokemon_id: integer ... 6 more fields]
Self battles: 0

```

Além disso, foram avaliadas possíveis duplicidades nos registros de combate. Como o conjunto de dados não possui informações temporais ou um identificador original da ocorrência, confrontos repetidos entre os mesmos Pokémon não foram automaticamente considerados duplicados inválidos, pois podem representar batalhas distintas.

As principais regras de consistência dos combates foram:

Validação	Regra esperada
Participantes	Identificadores válidos de Pokémon
Vencedor	Deve ser um dos dois participantes
Perdedor	Deve corresponder ao participante diferente do vencedor
Self-battle	Identificada e investigada separadamente
Registros repetidos	Investigados antes de qualquer possível remoção

5.4. Integridade Referencial

Além das validações realizadas individualmente em cada entidade, foi verificada a integridade referencial entre os dados de batalha e o cadastro de Pokémon. Os identificadores presentes nos campos **first_pokemon_id**, **second_pokemon_id** e **winner_id** foram comparados com os identificadores existentes na entidade Pokémon.

O objetivo dessa verificação foi garantir que todos os participantes e vencedores presentes no histórico de batalhas possuíssem um registro correspondente no cadastro utilizado pelo pipeline.

As verificações realizadas não identificaram chaves estrangeiras inválidas nos dados processados. Dessa forma, todos os identificadores utilizados nos registros de combate puderam ser relacionados a Pokémon existentes na base cadastral. Essa consistência permitiu construir posteriormente os relacionamentos da camada Gold sem necessidade de tratamento ou exclusão de registros devido a referências inexistentes.

```
20 hours ago (4s) 21 Python

#Verificando se há referências inválidas nos IDs
pokemon_ids = pokemon_silver.select("pokemon_id")

invalid_first = (
    combats_silver
    .join(
        pokemon_ids,
        combats_silver.first_pokemon_id
        == pokemon_ids.pokemon_id,
        "left_anti"
    )
)

invalid_second = (
    combats_silver
    .join(
        pokemon_ids,
        combats_silver.second_pokemon_id
        == pokemon_ids.pokemon_id,
        "left_anti"
    )
)

invalid_winner_fk = (
    combats_silver
    .join(
        pokemon_ids,
        combats_silver.winner_id
        == pokemon_ids.pokemon_id,
        "left_anti"
    )
)

print("First inválidos:", invalid_first.count())
print("Second inválidos:", invalid_second.count())
print("Winner inválidos:", invalid_winner_fk.count())

> See performance \(3\) ✓ Checked for improvements

> 4 dataframes: invalid_first invalid_second invalid_winner_fk pokemon_ids
First inválidos: 0
Second inválidos: 0
Winner inválidos: 0
```

5.5. Qualidade da Camada Gold

Após a modelagem das tabelas Gold, foram realizadas validações adicionais para garantir a consistência das métricas produzidas pelo pipeline.

Na tabela `agg_pokemon_performance`, foi verificado que, para cada Pokémon, a quantidade total de batalhas corresponde à soma entre vitórias e derrotas:

total_battles = total_wins + total_losses

Também foi verificado que **win_rate** permanece dentro do intervalo entre 0 e 1 para Pokémon que possuem participações registradas.

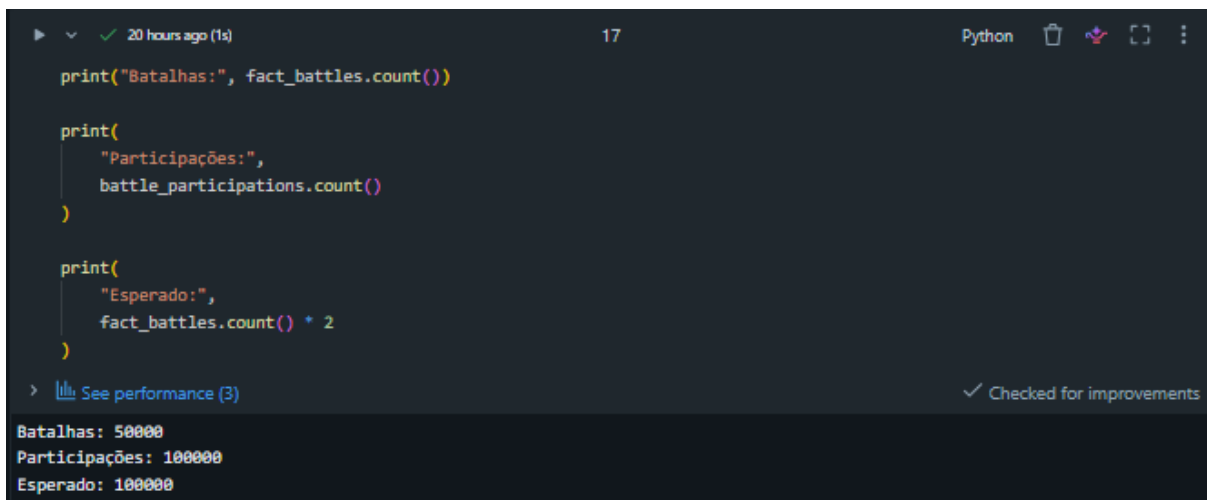
No nível global das batalhas, duas propriedades adicionais foram utilizadas para verificar a consistência da transformação.

Cada batalha deve produzir exatamente **uma vitória**, portanto:

Total de vitórias = Total de batalhas

Além disso, como cada confronto possui dois participantes:

Total de participações = 2 × Total de batalhas



```
print("Batalhas:", fact_battles.count())

print(
    "Participações:",
    battle_participations.count()
)

print(
    "Esperado:",
    fact_battles.count() * 2
)
```

> [See performance \(3\)](#) ✓ Checked for improvements

Batalhas: 50000
Participações: 100000
Esperado: 100000

Por fim, foi verificada a correspondência entre a quantidade de registros da dimensão `dim_pokemon` e da tabela `agg_pokemon_performance`. Essa validação garante que todos os Pokémon cadastrados permaneçam representados na estrutura agregada, inclusive aqueles sem registros históricos de batalha.

Essas verificações permitem validar não apenas os dados recebidos na origem, mas também as transformações e agregações produzidas pelo próprio pipeline.

5.6. Resultados das Validações

As verificações realizadas demonstraram que os dados de origem apresentavam consistência suficiente para que não fosse necessária a exclusão de registros durante a construção da camada Silver.

A quantidade de registros foi mantida entre as camadas Bronze e Silver, indicando que as transformações realizadas nessa etapa tiveram caráter predominantemente estrutural e de padronização, e não corretivo.

Entre os principais resultados observados, destacam-se:

- manutenção da quantidade de registros entre Bronze e Silver;
- ausência de vencedores que não pertenciam aos respectivos confrontos;
- ausência de referências inválidas entre os identificadores presentes nos combates e o cadastro de Pokémon;
- preservação dos valores nulos de **type_2**, considerados válidos de acordo com o significado do atributo;
- consistência entre o total de batalhas, vitórias e derrotas na camada Gold;
- taxas de vitória dentro do domínio esperado;
- correspondência de uma vitória e duas participações para cada batalha.

Esses resultados não significam que as camadas Bronze e Silver sejam equivalentes. Embora a quantidade de registros tenha sido preservada, a camada Silver apresenta uma

estrutura padronizada, tipada e previamente submetida às regras de qualidade definidas para o projeto.

A aplicação dessas validações possibilitou que a camada Gold fosse construída sobre dados previamente verificados, reduzindo a necessidade de repetir tratamentos de qualidade durante as consultas analíticas.

6. Análise dos Dados

6.1. Metodologia da Análise

Após a conclusão das etapas de ingestão, tratamento, validação e modelagem, os dados disponibilizados na camada Gold foram utilizados para responder às perguntas de negócio estabelecidas no início do projeto. As análises foram realizadas principalmente por meio de consultas SQL no Databricks, utilizando `dim_pokemon`, `fact_battles` e `agg_pokemon_performance` como fontes.

A utilização exclusiva da camada Gold permite separar o consumo analítico das etapas anteriores de preparação dos dados. Dessa forma, as consultas não precisam acessar diretamente os arquivos CSV ou reproduzir tratamentos realizados nas camadas Bronze e Silver.

Para cada pergunta foram utilizadas métricas adequadas ao objetivo da análise, como taxa de vitória, médias dos atributos e número de participações. Os resultados foram complementados por visualizações produzidas no próprio ambiente Databricks.

6.2. Distribuição do Número de Batalhas

Antes da construção do ranking de Pokémon por taxa de vitória, foi analisada a distribuição da quantidade de batalhas disputadas por cada Pokémon.

Essa análise é necessária porque a utilização isolada da taxa de vitória poderia produzir resultados pouco representativos. Um Pokémon com poucas participações poderia, por exemplo, apresentar uma taxa elevada mesmo sem possuir uma quantidade suficiente de observações para comparação com Pokémon que participaram de um número maior de confrontos.

A partir da distribuição observada, foi estabelecido um mínimo de **127 batalhas**, valor definido a partir da média de participações por Pokémon no conjunto de dados.

Esse limite foi posteriormente aplicado à análise de ranking, permitindo comparar Pokémon com uma quantidade relevante de observações e reduzindo a influência de taxas elevadas provenientes de poucas participações.

19 hours ago (10s) 3 SQL

```
%sql
SELECT
  MIN(total_battles) AS min_battles,
  ROUND(AVG(total_battles), 2) AS avg_battles,
  percentile_approx(total_battles, 0.25) AS p25,
  percentile_approx(total_battles, 0.50) AS median,
  percentile_approx(total_battles, 0.75) AS p75,
  MAX(total_battles) AS max_battles
FROM mvp_data_engineering.gold.agg_pokemon_performance
WHERE total_battles > 0;
```

> See performance (1) ✓ Checked for improvements

	1 st min_battles	1.2 avg_battles	1 st p25	1 st median	1 st p75	1 st max_battles
1	92	127.55	120	128	135	164

6.3. Pergunta 1 — Speed × Taxa de Vitória

Pergunta: Pokémon com maior atributo de velocidade (**Speed**) apresentam maior taxa de vitória?

Para investigar a relação entre velocidade e desempenho, os Pokémon foram analisados considerando seu atributo **speed** e sua respectiva taxa de vitória.

Os resultados indicaram uma **associação positiva entre Speed e taxa de vitória**, sugerindo que Pokémon com maior velocidade tendem a apresentar melhor desempenho nos combates presentes no conjunto de dados. Esse resultado também é consistente com a comparação entre os atributos médios de vencedores e perdedores apresentada na análise seguinte, na qual Speed apresentou a maior diferença entre os dois grupos.

Entretanto, a associação observada não deve ser interpretada como uma relação causal. O resultado demonstra uma relação existente no conjunto de dados analisado, enquanto outros atributos e características dos Pokémon também podem contribuir para o desempenho.

19 hours ago (3s) 6

```
%sql

WITH speed_groups AS (
  SELECT
    p.pokemon_id,
    p.name,
    p.speed,
    a.total_battles,
    a.win_rate,
    CASE
      WHEN p.speed < 40 THEN '01 - < 40'
      WHEN p.speed < 60 THEN '02 - 40-59'
      WHEN p.speed < 80 THEN '03 - 60-79'
      WHEN p.speed < 100 THEN '04 - 80-99'
      ELSE '05 - >= 100'
    END AS speed_group
  FROM mvp_data_engineering.gold.dim_pokemon p
  INNER JOIN mvp_data_engineering.gold.agg_pokemon_performance a
    ON p.pokemon_id = a.pokemon_id
  WHERE a.total_battles > 0
)

SELECT
  speed_group,
  COUNT(*) AS pokemon_count,
  ROUND(AVG(speed), 2) AS avg_speed,
  ROUND(AVG(win_rate) * 100, 2) AS avg_win_rate_pct
FROM speed_groups
GROUP BY speed_group
ORDER BY speed_group;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	^A _C speed_group	¹ ₃ pokemon_count	¹ ₂ avg_speed	¹ ₂ avg_win_rate_pct
1	01 - < 40	127	27.6	17.44
2	02 - 40-59	186	48.37	30.07
3	03 - 60-79	178	66.99	49.8
4	04 - 80-99	157	87.96	68.47
5	05 - >= 100	136	112.94	87.12

5 rows 2.696s runtime Refreshed 19h ago

ⓘ This result is stored as `_sqlidf` and can be used in other [Python](#) and [SQL](#) cells.

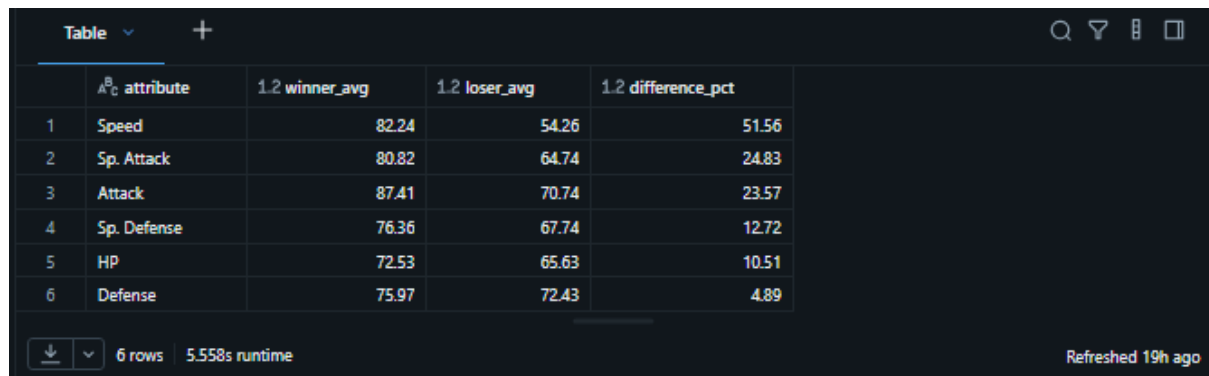
6.4. Pergunta 2 — Atributos de Vencedores × Perdedores

Pergunta: Quais atributos apresentam as maiores diferenças entre Pokémon vencedores e perdedores?

Para identificar quais características mais diferenciam vencedores e perdedores, foram calculadas as médias dos seis principais atributos de batalha para os dois grupos. Os vencedores apresentaram médias superiores em todos os atributos analisados. A maior diferença foi observada em **Speed**, cuja média foi 51,56% superior entre os vencedores.

Na sequência aparecem **Sp. Attack**, com diferença de 24,83%, e **Attack**, com 23,57%. Em contrapartida, **Defense** apresentou a menor diferença entre os grupos, com 4,89%.

Os resultados reforçam a velocidade como o atributo que mais distingue vencedores e perdedores dentro do histórico de batalhas analisado. A consulta ficou um pouco grande, por conta disso, o print será da tabela gerada.



	attribute	1.2 winner_avg	1.2 loser_avg	1.2 difference_pct
1	Speed	82.24	54.26	51.56
2	Sp. Attack	80.82	64.74	24.83
3	Attack	87.41	70.74	23.57
4	Sp. Defense	76.36	67.74	12.72
5	HP	72.53	65.63	10.51
6	Defense	75.97	72.43	4.89

6 rows 5.558s runtime Refreshed 19h ago

6.5. Pergunta 3 — Legendary × Não Legendary

Pergunta: Pokémon classificados como Legendary apresentam taxa de vitória diferente dos Pokémon não Legendary?

Para realizar essa comparação, as participações em batalhas foram classificadas de acordo com a condição **is_legendary** de cada Pokémon. Os resultados demonstram uma diferença relevante entre os dois grupos. Enquanto os Pokémon não Legendary apresentaram taxa de vitória de **47,51%**, os Legendary alcançaram **77,88%**. Isso representa uma diferença de **30,37 pontos percentuais** entre as taxas de vitória.

Dessa forma, nos combates presentes no conjunto de dados, Pokémon classificados como Legendary apresentaram desempenho superior aos demais.

12:02 AM (4s)

13

%sql

WITH participations AS (

SELECT

first_pokemon_id AS pokemon_id,

CASE

WHEN winner_id = first_pokemon_id THEN 1

ELSE 0

END AS win

FROM mvp_data_engineering.gold.fact_battles

UNION ALL

SELECT

second_pokemon_id AS pokemon_id,

CASE

WHEN winner_id = second_pokemon_id THEN 1

ELSE 0

END AS win

FROM mvp_data_engineering.gold.fact_battles

)

SELECT

p.is_legendary,

COUNT(*) AS battle_participations,

SUM(pa.win) AS wins,

ROUND(

SUM(pa.win) / COUNT(*) * 100,

2

) AS win_rate_pct

FROM participations pa

INNER JOIN mvp_data_engineering.gold.dim_pokemon p

ON pa.pokemon_id = p.pokemon_id

GROUP BY p.is_legendary

ORDER BY p.is_legendary;

See performance (1)

Checked for improvements

Table

+

Q F I □

	is_legendary	1 battle_participations	1 wins	1.2 win_rate_pct
1	false	91812	43623	47.51
2	true	8188	6377	77.88

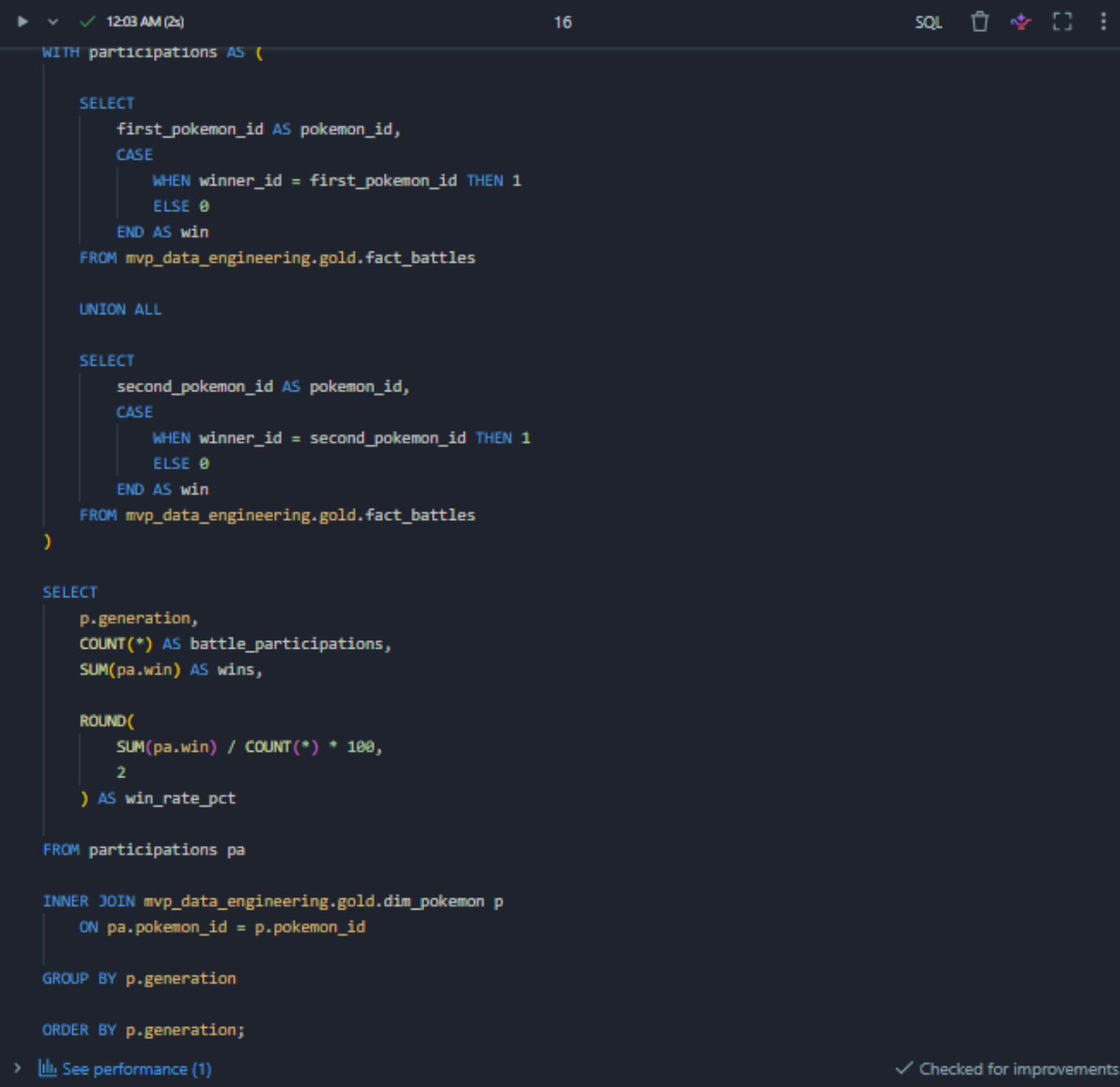
6.6. Pergunta 4 — Taxa de Vitória por Geração

Pergunta: Como a taxa de vitória varia entre as diferentes gerações de Pokémon?

As participações foram agrupadas de acordo com a geração de cada Pokémon para calcular a taxa de vitória correspondente. A **4ª geração apresentou a maior taxa de vitória, com 53,91%**, seguida pelas gerações 1 e 5, com 51,76% e 51,74%, respectivamente. A **2ª geração apresentou a menor taxa de vitória, com 44,82%**.

36

Apesar dessas diferenças, não foi observado um padrão crescente ou decrescente entre as gerações. Dessa forma, os resultados não indicam que pertencer a uma geração mais recente esteja necessariamente associado a uma maior taxa de vitória.



```

WITH participations AS (
    SELECT
        first_pokemon_id AS pokemon_id,
        CASE
            WHEN winner_id = first_pokemon_id THEN 1
            ELSE 0
        END AS win
    FROM mvp_data_engineering.gold.fact_battles

    UNION ALL

    SELECT
        second_pokemon_id AS pokemon_id,
        CASE
            WHEN winner_id = second_pokemon_id THEN 1
            ELSE 0
        END AS win
    FROM mvp_data_engineering.gold.fact_battles
)

SELECT
    p.generation,
    COUNT(*) AS battle_participations,
    SUM(pa.win) AS wins,

    ROUND(
        SUM(pa.win) / COUNT(*) * 100,
        2
    ) AS win_rate_pct

FROM participations pa

INNER JOIN mvp_data_engineering.gold.dim_pokemon p
    ON pa.pokemon_id = p.pokemon_id

GROUP BY p.generation

ORDER BY p.generation;

```

See performance (1) ✓ Checked for improvements

	1 ² generation	1 ² battle_participations	1 ² wins	1.2 win_rate_pct
1	1	20311	10513	51.76
2	2	13310	5966	44.82
3	3	20388	9708	47.62
4	4	15137	8160	53.91
5	5	20607	10662	51.74
6	6	10247	4991	48.71

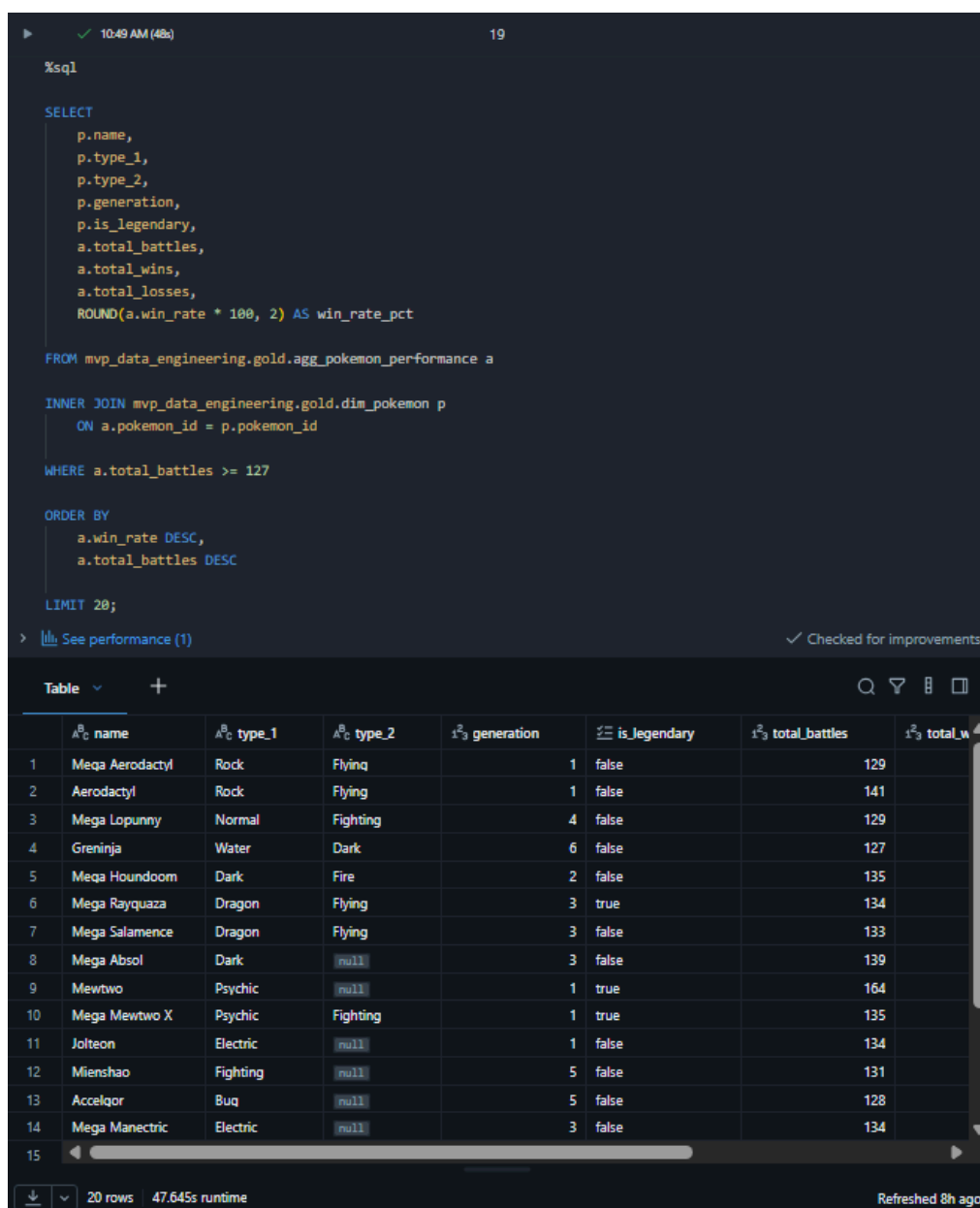
6.7. Pergunta 5 — Pokémon com Maiores Taxas de Vitória

Pergunta: Quais Pokémon apresentam as maiores taxas de vitória considerando um número mínimo relevante de batalhas?

Para evitar que Pokémon com poucas participações ocupassem as primeiras posições devido a uma quantidade reduzida de observações, foi aplicado o limite mínimo de **127 batalhas**, definido anteriormente a partir da média de participações.

Considerando esse critério, o **Mega Aerodactyl** apresentou a maior taxa de vitória, com **98,45%**, totalizando 127 vitórias em 129 batalhas. Entre as posições seguintes aparecem **Aerodactyl (96,45%)**, **Mega Lopunny (96,12%)** e **Greninja (96,06%)**.

A aplicação do número mínimo de batalhas torna o ranking mais representativo ao restringir a comparação a Pokémon que possuem uma quantidade relevante de participações no conjunto de dados.



```
%sql

SELECT
  p.name,
  p.type_1,
  p.type_2,
  p.generation,
  p.is_legendary,
  a.total_battles,
  a.total_wins,
  a.total_losses,
  ROUND(a.win_rate * 100, 2) AS win_rate_pct
FROM mvp_data_engineering.gold.agg_pokemon_performance a
INNER JOIN mvp_data_engineering.gold.dim_pokemon p
  ON a.pokemon_id = p.pokemon_id
WHERE a.total_battles >= 127

ORDER BY
  a.win_rate DESC,
  a.total_battles DESC

LIMIT 20;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	name	type_1	type_2	generation	is_legendary	total_battles	total_w
1	Mega Aerodactyl	Rock	Flying	1	false	129	
2	Aerodactyl	Rock	Flying	1	false	141	
3	Mega Lopunny	Normal	Fighting	4	false	129	
4	Greninja	Water	Dark	6	false	127	
5	Mega Houndoom	Dark	Fire	2	false	135	
6	Mega Rayquaza	Dragon	Flying	3	true	134	
7	Mega Salamence	Dragon	Flying	3	false	133	
8	Mega Absol	Dark	null	3	false	139	
9	Mewtwo	Psychic	null	1	true	164	
10	Mega Mewtwo X	Psychic	Fighting	1	true	135	
11	Jolteon	Electric	null	1	false	134	
12	Mienshao	Fighting	null	5	false	131	
13	Accelgor	Bug	null	5	false	128	
14	Mega Manectric	Electric	null	3	false	134	
15							

20 rows 47.645s runtime Refreshed 8h ago

6.8. Discussão Geral dos Resultados

A análise dos dados históricos de batalhas permitiu identificar padrões relevantes relacionados ao desempenho dos Pokémon.

De forma geral, os resultados apontam que características individuais possuem relação importante com a taxa de vitória, com destaque para o atributo **Speed**. Além da associação positiva observada entre velocidade e taxa de vitória, a comparação direta entre vencedores e perdedores mostrou que Speed apresentou a maior diferença entre os dois grupos, sendo, em média, **51,56% superior entre os vencedores**. Outros atributos ofensivos, como Sp. Attack e Attack, também apresentaram diferenças relevantes, de 24,83% e 23,57%, respectivamente.

A classificação como **Legendary** também apresentou uma diferença expressiva de desempenho. Pokémon Legendary registraram taxa de vitória de **77,88%**, enquanto os demais apresentaram 47,51%. Esse resultado indica uma forte associação entre essa característica e o desempenho nos combates presentes no conjunto de dados.

Em relação às gerações, as diferenças foram mais moderadas. A **4ª geração apresentou a maior taxa de vitória, com 53,91%**, enquanto a 2ª geração registrou a menor, com 44,82%. Entretanto, não foi identificado um padrão consistente de crescimento ou redução do desempenho ao longo das gerações.

Por fim, considerando apenas Pokémon com pelo menos **127 batalhas**, o **Mega Aerodactyl apresentou a maior taxa de vitória, com 98,45%**, seguido por Aerodactyl, Mega Lopunny e Greninja. A aplicação desse limite mínimo tornou o ranking mais representativo ao reduzir a influência de Pokémon com poucas participações.

Em conjunto, os resultados destacam principalmente **velocidade, atributos ofensivos e a condição de Legendary** como características associadas a um melhor desempenho no conjunto de batalhas analisado.

Além dos resultados analíticos, esta etapa demonstra a finalidade da infraestrutura construída ao longo do MVP: os dados inicialmente disponibilizados em arquivos CSV foram ingeridos, tratados, validados e modelados até se tornarem estruturas analíticas capazes de responder diretamente às perguntas de negócio por meio da camada Gold.

7. Autoavaliação

7.1. Resultados Alcançados

O objetivo principal deste MVP foi desenvolver um pipeline de dados ponta a ponta em ambiente cloud, capaz de transformar os arquivos brutos do Pokémon Challenge em estruturas tratadas, validadas e preparadas para consumo analítico.

Considera-se que esse objetivo foi alcançado por meio da implementação de uma arquitetura Medallion no Databricks, contemplando as camadas Bronze, Silver e Gold.

Na camada Bronze, os arquivos de origem foram ingeridos e persistidos em formato Delta, preservando sua estrutura original e adicionando metadados de rastreabilidade. Na Silver, os dados foram padronizados, tipados e submetidos a diferentes verificações de qualidade e integridade. Por fim, na Gold, foram construídas estruturas direcionadas ao consumo analítico, incluindo uma dimensão de Pokémon, uma tabela fato de batalhas e uma tabela agregada de desempenho.

A partir dessas estruturas foi possível responder às cinco perguntas estabelecidas no início do projeto utilizando consultas sobre a camada Gold, sem necessidade de acessar diretamente os arquivos CSV ou reproduzir os tratamentos realizados nas etapas anteriores.

Além dos resultados analíticos obtidos, o desenvolvimento do MVP permitiu aplicar conceitos de Engenharia de Dados relacionados à ingestão, arquitetura em camadas, transformação, qualidade, modelagem, linhagem e disponibilização dos dados para consumo.

7.2. Dificuldades Encontradas

Durante o desenvolvimento foram encontradas algumas dificuldades técnicas e de modelagem que exigiram adaptações na implementação inicialmente planejada.

Uma das primeiras dificuldades esteve relacionada aos nomes das colunas presentes no arquivo `pokemon.csv`. Campos como **Type 1**, **Sp. Atk** e **Sp. Def** possuem espaços e caracteres especiais que exigiram tratamento específico durante a manipulação dos dados com Spark e durante sua persistência no formato Delta.

Para preservar a estrutura original na camada Bronze, foi necessário utilizar referências apropriadas às colunas durante as operações com PySpark e habilitar o recurso de *Column Mapping* na tabela Delta. Posteriormente, na camada Silver, os nomes foram padronizados para **snake_case**, simplificando sua utilização nas demais etapas.

Outro desafio esteve relacionado à modelagem dos registros de batalha. O arquivo de origem informa apenas os dois participantes e o vencedor, tornando necessário derivar informações como o Pokémon perdedor e criar um identificador técnico para cada batalha.

A construção das métricas de performance também exigiu uma transformação adicional. Como cada registro da tabela de combates representa uma batalha envolvendo dois Pokémon, foi necessário transformar cada confronto em duas participações antes da agregação. Essa estrutura permitiu calcular corretamente o número de batalhas, vitórias, derrotas e a taxa de vitória de cada Pokémon.

Por fim, as verificações de qualidade exigiram diferenciar possíveis problemas de dados de características legítimas do conjunto. Valores nulos em **type_2**, por exemplo, não representam ausência indevida de informação, uma vez que existem Pokémon que

possuem apenas um tipo. Da mesma forma, valores extremos nos atributos não foram automaticamente removidos, pois podem representar características válidas dos próprios Pokémon.

Essas dificuldades contribuíram para compreender que o processo de Engenharia de Dados não se limita à transformação técnica dos registros, exigindo também conhecimento do significado dos dados e definição explícita das regras utilizadas em cada etapa.

7.3. Limitações do Projeto

Apesar de atender aos objetivos definidos, o projeto apresenta limitações relacionadas principalmente às características do conjunto de dados utilizado.

O histórico de batalhas não possui informações temporais, impossibilitando análises relacionadas à evolução do desempenho ao longo do tempo. Também não estão disponíveis informações adicionais sobre o contexto de cada confronto, como condições da batalha, estratégias utilizadas ou outros fatores que poderiam influenciar o resultado.

Consequentemente, as associações identificadas nas análises devem ser interpretadas dentro dos limites do conjunto de dados. Por exemplo, a associação observada entre Speed e taxa de vitória não permite concluir isoladamente que a velocidade seja a causa de um maior desempenho, uma vez que outros atributos dos Pokémon podem contribuir simultaneamente para o resultado.

Outra limitação está relacionada ao próprio escopo do MVP. Os dados são provenientes de arquivos estáticos e, portanto, o pipeline não contempla atualmente mecanismos de ingestão incremental ou atualização contínua da fonte.

Além disso, embora a arquitetura tenha sido organizada em diferentes camadas e notebooks, a execução das etapas não foi implementada por meio de uma ferramenta dedicada de orquestração. A sequência Bronze → Silver → Gold depende da execução dos notebooks na ordem estabelecida pelo projeto.

Essas limitações não impedem as análises propostas, mas representam oportunidades de evolução para uma implementação de maior escala ou mais próxima de um ambiente produtivo.

7.4. Trabalhos Futuros

Como evolução do projeto, uma das principais possibilidades seria utilizar a infraestrutura desenvolvida neste MVP como fonte de dados para aplicações de Machine Learning. Nesse cenário, a camada Gold poderia disponibilizar uma estrutura específica de atributos (*features*) para treinamento e inferência de modelos preditivos de batalhas, permitindo integrar os dois projetos em uma arquitetura mais completa:

Fonte de Dados → Bronze → Silver → Gold → Machine Learning

Além dessa integração, outras evoluções possíveis incluem a implementação de ingestão incremental, automatização e orquestração da execução dos notebooks, criação de monitoramento das regras de qualidade e disponibilização das métricas por meio de dashboards.

Também seria possível expandir o modelo de dados caso novas fontes fossem incorporadas, adicionando informações relacionadas a movimentos, habilidades, características das batalhas ou outras variáveis capazes de enriquecer as análises e os modelos preditivos.

7.5. Considerações Finais

O desenvolvimento deste MVP permitiu construir uma solução completa de Engenharia de Dados a partir de um conjunto de arquivos originalmente utilizado para um projeto de Data Science. Ao longo do trabalho, os dados passaram por etapas de ingestão, armazenamento, padronização, validação, modelagem e análise, resultando em uma camada Gold estruturada para consumo analítico.

A utilização da arquitetura Medallion permitiu separar claramente as responsabilidades de cada etapa do pipeline: preservação dos dados de origem na Bronze, tratamento e validação na Silver e disponibilização de estruturas orientadas ao consumo na Gold.

As análises realizadas demonstraram que a arquitetura construída é capaz de transformar dados brutos em informações que respondem diretamente às perguntas definidas no início do projeto. Ao mesmo tempo, a estrutura desenvolvida permite a realização de novas análises sem necessidade de modificar as etapas anteriores do pipeline.

Como principal aprendizado, o projeto evidenciou a importância de tratar a Engenharia de Dados não apenas como um processo de movimentação e transformação de informações, mas como a construção de uma infraestrutura que garanta organização, qualidade, rastreabilidade e disponibilidade dos dados para seus consumidores. Dessa forma, o MVP estabelece uma base que pode ser utilizada tanto para análises quanto para futuras aplicações de Machine Learning.